Titre du sujet
Projet de programmation 2

Les membres du groupe
L3 informatique
Faculté des Sciences
Université de Montpellier.

2021-2022

Résumé

Description très succinte du problème et des différentes étapes de réalisation

Table des matières

1 Présentation du Sujet **3**

2 Développements Logiciel : Conception, Modélisation, Implémentation **3**

3 Algorithmes et Structures de Données **6**

4 Analyse des résultats **7**

5 Gestion du Projet **7**

6 Bilan et Conclusions **7**

1 Présentation du Sujet

1-2 pages

- Présenter le problème étudié et le contexte dans lequel le projet se positionne.
  - projet en licence informatique L3
  - méthode de Jos Stam "Stable FLuids"
  - simulation d'un fluide en temps réel
  - intéraction utilisateur
- Motiver l'intérêt du problème étudié par rapport à votre parcours d'études et au monde de l'informatique.

  Nous avons choisi ce projet car il nous a parru être le plus adapté à nos compétences, étant donné notre parcours en double licence mathématiques et informatique. En effet, nous avons rapidement compris les concepts de base de la simulation, ce qui nous a permis d'implémenter la méthode de Jos Stam "Stable Fluids". Nous avons également aimé l'idée d'implémenter une interface graphique pour visualiser en temps réél l'évolution de notre projet et le fait de pouvoir intéragir avec celui-ci. De plus, c'est avec ce projet que nous avons découvert le langage c++ et la bibliothèque OpenGl, qui sont des outils etentiels pour la simulaiton de fluides mais aussi dans de nombreux domaines tels que les jeux vidéos, la météorologie ou l'environnment.
- Présenter les différentes approches possibles pour la résolution du problème, et en particulier celle choisie.

  Pour implémenter la simulation de fluides en 2D, 2 méthodes sont possibles : la méthode eulérienne ou la méthode lagrangienne. La méthode eulérienne consiste à représenter le fluide dans une grille fixe, dans laquelle on stocke les différentes propriétés du fluide, telles que la densité, la chaleur, la vitesse, etc. La méthode lagrangienne consiste à représenter le fluide à l'aide de particules, c'est à dire que chaque particule contient les propriétés du fluide donc celui-ci est représenté comme à l'aide de particules. Pour l'implémentation de ce projet, nous avons choisi la méthode eulérienne, qui est plus simple à implémeenter et moins couteuse en complexité. La méthode lagrangienne peut rapidement exploser en temps de calcul en fonction du nombre de particules.
- Donner le cahier des charges détaillé
  - Partie 2 du cahier des charges de base

2 Développements Logiciel : Conception, Modélisation, Implémentation

8 à 15 pages

1. Technologies utilisées
   - langage de programmation : c++ (justification pour l'optimsation, etc)
   - openGL (affichage : api graphique)
   - Imgui (facile d'utilisation)
2. Présenter les développements logiciel réalisés dans le cadre du projet. (fenetre, zone de rendu et panneau de parametres, et zone de rendu (shader, openGL))

   Dans cette section, nous allons présenter l'évolution de l'affichage de notre projet au cours du semestre, depuis les premiers affichages (grille de chaleur) jusqu'à la version finale de notre simulation de fluide. Nous nous intéresserons successivement aux différentes étapes de notre projet, grille de chaleur, affichage du fluide, fenêtre de paramètres et ajouts d'obstacles.

Évolution 1 : premier affichage d'une grille de chaleur (voir figure1)

Dans notre première version, l'objectif était d'avoir un premier affichage graphique avec OpenGL. Nous avons donc initialisé une fenêtre et une zone de rendu OpenGL affichant une grille. Cette grille prend des valeurs et les affiche sous forme de carte de chaleur. Notons la pertinence des couleurs, nous avons utilisé plusieurs teintes de rouges pour garder une cohérence avec la diffusion de chaleur (à coté d'une case très chaude se trouve une case un peu moins chaude mais pas froide,

donc moins rouge mais pas bleue). Cette étape nous a permis de tester la création de la fenêtre, la boucle principale et l'utilisation de shaders pour colorer chaque cellule en fonction d'une valeur.

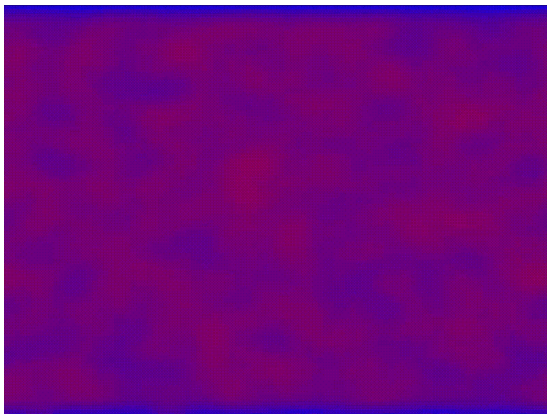

FIGURE 1 – Première version : affichage d'une grille de chaleur.

Évolution 2 : première simulation de fluide (voir figure2)

Dans un second temps, nous avons implémenté la méthode de Jos Stam *Real-Time Fluid Dynamics for Games* qui nous a permit d'avoir un beau rendu en 2D et une première simulation de fluide. Cette méthode repose sur une représentation du fluide dans une grille Eulerienne (voir section ....). La zone de rendu affiche donc la densité du fluide qui est mise à jour à chaque itération de la boucle de simulation. Dans cette version, les paramètres (couleur, densité, intensité) sont modifiables uniquement dans le code lui-même, c'est-à-dire qu'ils sont hard-codés. Cette version est intéressante car elle nous permet d'observer une simulation de fluide en temps réel.

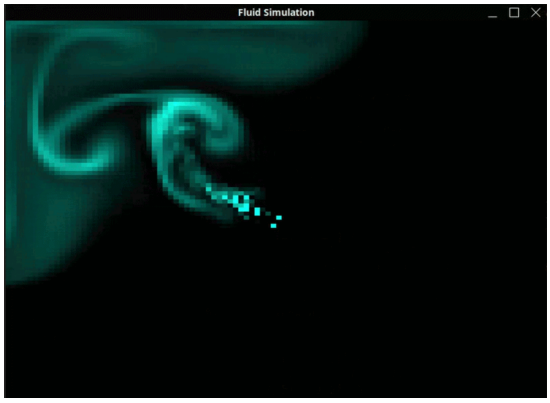

FIGURE 2 – Deuxieme version : affichage d'un fluide.

Évolution 3 : ajout du panneau de paramètres (voir figure3)

Nous avons ensuite ajouté un panneau de paramètres en utilisant la bibliothéque ImGui. Ce panneau nous permet de modifier en temps réel, lors de la simulation, différents paramètres tels que la densité, la couleur ou encore la direction du fluide. Les valeurs saisies par l'utilisateur sont envoyées au module de simulation qui met à jour la grille. Notons que ce panneau peut s'enlever et se remettre en pressant la touche "h".

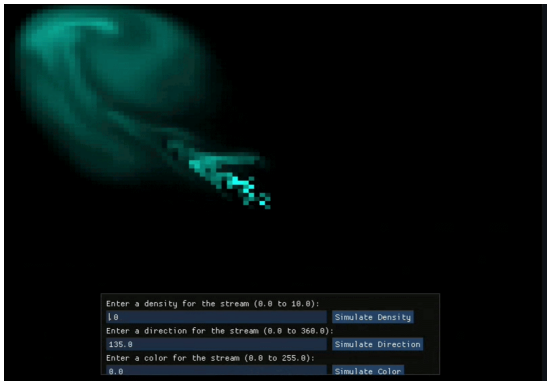

FIGURE 3 – Troisième version : ajout d'un panneau de paramètres.

Évolution 4 : gestion des obstacles

Enfin nous avons étoffé notre simulation en ajoutant des obstacles. Nous avons donc rajouté au panneau de paramètres les différents outils pour contrôler ceux-ci. En effet, l'utilisateur peut activer ou désactiver les obstacles et changer leur forme, nous pouvons également rajouter plusieurs obstacles. Lors de la simulation, nous pouvons les déplacer avec le curseur de notre souris. Nous avons également implémenté deux modes d'injection du fluide : un point ou une ligne qui modifient le comportement du fluide. Par la même occasion nous avons rajouté un mode d'affichage représentant la chaleur du fluide, l'utilisateur choisi sur le panneau de paramètres si le fluide sort chaud ou froid (rouge ou bleu).

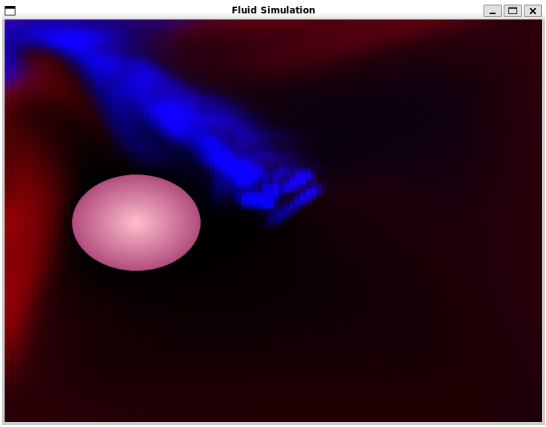

FIGURE 4 – Quatrième version : ajout d'un obstacle.

3. Présenter les principaux modules du logiciel développé dans le cadre du projet. Utiliser le langage UML pour la modélisation : donner le diagramme de cas d'utilisation et le diagramme des classes.
   — diagramme UML (cellules, grille, etc)
   — PARTIE A : terminé, bien implémenté
   — grille de départ (axel) (description des modules dans l'état actuel/final du projet)
   — obstacle (satine)
   — chaleur (saurane)
   — point ou ligne (gaelle)
   — calcul d'une simulation, le fluide, physique
   — PARTIE B : pas terminé
   — bluetooth, lagrangien, musique, etc
4. Décrire les fonctionnalités de l'interface graphique implémentée (champs de simulation, bouton, affichage de l'obstacle).

— description des boutons et des interactions possibles

5. Décrire le format des données en entrée ou encore les conventions utilisées pour les entrées de vos programmes.(représentation du fluide, grille, stackage des paramètres)
   — fluide stocké dans une grille
   — grille composée de cellules
   — cellules contiennent les paramètres de la simulation (densité, chaleur, vitesse)
   — expliquer ce que représentent ces paramètres
6. Statistiques du projet

   En termes de statistiques, notre projet est composé de certaines classes principales : Cellule, Grille, Obstacle, etc. Il est composé de 22 fichiers (cpp et h), et contient plus de 2000 lignes de code. Le projet est organisé en plusieurs modules, tels que le module de gestion des obstacles, le module de gestion de la chaleur, etc.

3 Algorithmes et Structures de Données

2 à 3 pages

1. Présenter les principaux algorithmes implémentés. En illustrer le fonctionnement avec des exemples. Attention : il s'agit ici de choisir 1 ou 2 algorithmes intéressants, et non pas de présenter tous les algorithmes implémentés.

L'Advection

La Diffusion

La Projection

Après avoir appliqué les algorithmes d'advection et de diffusion, l'algorithme de projection implémente la décomposition de Hodge. C'est un algorithme qui permet de rendre le champ de vitesse du fluide incompressible, Cet algorithme force la vitesse à être conservatrice de masse, c'est à dire que le fluide ne peut pas être créé ou détruit. C'est une propriété importante pour une simulation réaliste. Visuellement, la projection force l'écoulement à avoir de meilleures courbes, et évite les écoulements en ligne droite.
L'algorithme de projection est composé de 3 étapes :
— calcul de la divergence : pour chaque cllule de la grille, on évalue la divergence discrète :

$$div_{i,j} = -\frac{h}{2} left(u_{i+1,j} - u_{i-1,j} ight) + left(v_{i,j+1} - v_{i,j-1} ight) \tag{1}$$

où $h = 1/N$ est la taille d'une cellule de la grille, et u et v sont les composantes de la vitesse du fluide.
— résolution de l'équation de Poisson :
On résout itérativement l'équation $\nabla^2 p = div$ par 20 itérations de relaxation de Gauss-Seidel, où p est un champ de pression :

$$p_{i,j}^{(k+1)} = \frac{1}{4} left(div_{i,j} + p_{i+1,j}^{(k)} + p_{i-1,j}^{(k)} + p_{i,j+1}^{(k)} + p_{i,j-1}^{(k)} - div_{i,j} ight) \tag{2}$$

On réutilisa exactement le code de la diffusion pour assurer la cohérence du code et éviter les redondances.
— correction du champ de vitesse :
pour chaque cellule de la grille, on corrige la vitesse en soustrayant le gradient du champ de pression :

$$mathbf u'_{i,j} = \mathbf{u}_{i,j} - \frac{1}{2h} \left(p \right)_{i+1,j} - p_{i-1,j} \tag{3}$$

2. Évaluer la complexité théorique en temps des algorithmes présentés
   — complexité globale de update_simulation

4 Analyse des résultats

2 à 4 pages L'analyse expérimentale sera faites dans cette section. Elle sera plus ou moins importante dans votre projet comme pour les deux points précédent, cette section devra donc être développée en conséquence.

1. Illustrer les performances ainsi que l'efficacité du logiciel implémenté à l'aide de graphiques.
   — calcul de complexités en fonction de la taille de la grille, etc (plot sur python), nombre d'obstacles, étape d'avection/relaxation (changer les paramètres et comparer)
2. Analyser (et comparer, si plusieurs) les performances des solutions implémentées.
   — eulerien vs lagrangien (uniquement eulerien a été implémenté)
   — analyser les performances visuelles en fonction du nombre d'obstacles, densité, etc (mettre des screens)

5 Gestion du Projet

(1-2 pages)

1. Présenter la gestion du projet et les documents de planification rédigés (par exemple, le diagramme de Gantt).
   — répartition des roles
   — diagramme de gantt
   — branches github, commits
2. Discuter les changements majeurs effectués en cours de projet.
   — passage de la grille de départ à la nouvelle grille (pourquoi ?)
   — la façon d'implémenter les obstacles a été modifiée quand on est passé de l'affichage d'uun obstacle à plusieurs, est-ce que ça a impacté les autres modules ? pourquoi ? comment ?
   — implémentation de la fenetre pour rentrer les paramètresuu

6 Bilan et Conclusions

Indiquer les fonctionnalités mises en œuvre par rapport au cahier des charges de départ, les points ouvertes. Les perspectives du projet décrivent ce que l'on pourrait ajouter, modifier, réécrire afin d'améliorer le projet.

1. Fonctionnalités rélisées qui étaient présentent dans le cahier des charges
   — interface graphique avec une fenetre d'affichage du fluide et des curseurs
   — simulation du fluide en 2D
   — possibilité d'agir sur le fluide avec des curseurs
   — possibilité d'ajouter des obstacles
2. Fonctionnalités non rélisées qui étaient présentent dans le cahier des charges
   — possibilité de controler le fluide à distance via un smartphone (bluetooth)
   — intégrer un mode musique qui fait réagir de fluide en focntion du son
3. Perspecvtives du projet pour la suite : ce qu'on aurait pu implémenter avec plus de temps.
   — lagrangien
   — autres fonctionnalités possibles avec les obstacles, etc

Références

[1] Donald E. Knuth (1986) *The TEX Book*, Addison-Wesley Professional.

[2] Leslie Lamport (1994) *LATEX : a document preparation system*, Addison Wesley, Massachusetts, 2nd ed.